WEB APPLICATION DVELOPMENT-II

QUESTION BANK

MALLA REDDY VISHWAVIDYAPEETH
(Deemed to be University)
Hyderabad

MALLA REDDY TECHNICAL CAMPUS
MRTC

MODULE – I

S.No	Short Answer Questions	Page NO
1	Explain Briefly about Hoisting Process in JavaScript Programming with an Example?	
2	What is a Variable? Explain its scope of Variable with a simple Example?	
3	What problem does asynchronous JavaScript solve in single-threaded execution?	
4	How the 'this' Keyword will process in JavaScript Programming explain with an example?	
5	Explain the Types of Destructuring with Syntax in JavaScript Programming with simple Examples?	
6	Explain briefly about Object Destructuring in JavaScript Programming with an Example?	
7	What is the Syntax for Arrow Function in JavaScript Programing, Explain with an example?	

S.No	Long Answer Questions	Page no
1	Explain Default binding and Implicit binding of "this" keyword along with example program?	
2	How is Error Handling will handle in JavaScript Programing? Explain the Error handling Mechanisms?	
3	Explain briefly about Classes mechanism in JavaScript Programming in detail along with example?	
4	Write the Syntax and Program of any ten Array Methods in JavaScript Programming?	
5	Write the Syntax and Program of any ten String Methods in JavaScript Programming?	
6	Explain in detail about try block, catch block and finally block in JavaScript Programming along with example program?	

MODULE – II

S.No	Short Answer Questions	Page No.
1	What problem does asynchronous JavaScript solve in single-threaded execution?	
2	How Synchronous programming will process in JavaScript code Explain in detail ?	
3	Explain the Promise states in JavaScript Programming of following a) Pending state b) Fulfilled state c) Reject state	
4	Explain the methods that can resolve in promise of JavaScript Programing?	
5	Explain briefly about states in promise Asynchronous operation in JavaScript Programming?	
6	How Asynchronous programming will process in JavaScript code Explain in detail ?	

S.No	Long Answer Questions	Page No.
1	What is a Callback Function, and how does it enable asynchronous behaviour in JavaScript explain in detail with an Example?	
2	Demonstrate how to implement time delays in JavaScript programming with an example?	
3	How does overusing callbacks lead to unmaintainable code, and what are the solutions Explain in detail?	
4	How do JavaScript timing events (setTimeout and setInterval) work, what are their differences, and Explain their implementation with examples?	
5	Explain the concept of “Pyramid of Doom” in detail along with example program?	
6	How do Promises manage asynchronous operations, and what is their constructor structure and Explain briefly with an example?	

MODULE – III

S.No	Short Answer Questions	Page No.
1	How does JSX bridge the gap between HTML markup and JavaScript logic in React?	
2	Explain the core concepts of React and its significance in modern front-end development?	
3	What role does JSX play in React development, and how does it simplify component creation?	
4	Explain the role of the Virtual DOM in updating user interfaces efficiently?	
5	What are the differences between functional components and class components in React?	
6	Explain the role of props in React and why a child component cannot change its own props?	

S.No	Long Answer Questions	Page No.
1	What are the core principles and features of the React library Explain with an Example?	
2	What are the different ways to create components in React and when should I use them Explain with a suitable Example?	
3	Compare functional components and class components in React with examples.	
4	Explain the Component Lifecycle in detail, covering the Mounting, Updating, and Unmounting phases.	
5	What is JSX? Write a code snippet demonstrating how to embed JavaScript expressions and attributes within JSX?	
6	Describe the process of Props Communication between a parent and a child component with a code snippet?	

MODULE – IV

S.No	Short Answer Questions	Page No.
1	How do you add state management capabilities to a functional React component?	
2	What constitutes a side effect, and how can you handle it in functional components in React?	
3	What is the role of keys in React's reconciliation (diffing) algorithm?	
4	Explain the performance and functional tradeoffs between controlled and uncontrolled inputs in React?	
5	What are the cleanup steps for useEffect Hook to prevent memory leaks?	

S.No	Long Answer Questions	Page No.
1	Explain the useReducer hook and how it differs from useState for complex state logic?	
2	Demonstrate how to create and implement a Custom Hook to reuse logic across components?	
3	Describe the useEffect Hook in detail, specifically how the dependency array controls the execution of side effects?	
4	Discuss Conditional Rendering techniques in React (e.g., Ternary operators, Logical &&)?	
5	Discuss Event Handling Techniques in React. How do you pass arguments to an event handler without triggering it on render?	
6	What are the best practices for handling form submission and input data in React Explain with Suitable Example?	

MODULE – V

S.No	Short Answer Questions	Page No.
1	What is the role of BrowserRouter and Route in React Router?	
2	How do you extract values using the useParams() hook?	
3	Define the Context API and its role in global state management?	
4	What is Client-Server Communication in the context of a React app?	
5	What are Nested Routes, and when should they be used?	

S.No	Long Answer Questions	Page No.
1	Compare the Context API with the Reducer Pattern for managing advanced application state?	
2	Explain the implementation of Application Routing, including navigation components like Link and NavLink?	
3	Describe the general workflow for implementing User Authentication in a React application?	
4	Discuss Advanced State Management using the Context API to solve the "Prop Drilling" problem?	
5	Explain the Route Setup and Implementation process in React Router, including BrowserRouter, Routes, and Route?	
6	Explain the workflow of Client-Server Communication using fetch or Axios within a component's lifecycle?	

MODULE-I
Short Answer Questions

1. **Explain Briefly about Hoisting Process in JavaScript Programming with an Example?**
   Answer:

   Hoisting in JavaScript is a behaviour where declarations for functions, variables, classes, and imports are conceptually moved to the top of their containing scope during the compilation phase, before the code is executed.

   This allows some elements to be used before they are explicitly declared in the code, though their behaviour varies significantly based on how they are declared.

```
Example:

console.log(a); // using before declaring-- allowed but
undefined var a=10;
console.log(a);

output:

undefined
10
```

2. **What is a Variable? Explain its scope of Variable with a simple Example?**

Answer:

Global Scope:

- Variables declared outside of any function or block reside in the global scope.
- They are accessible from anywhere in the entire JavaScript program, including within functions and blocks.

Function Scope:

- They are only accessible within the function where they are declared and not outside of it.
- Each function creates its own scope, and variables declared within that function are local to it.

Block Scope:

- Introduced with ES6,variables declared with **let** and **const** inside a block (defined by curly braces {} like in if statements, for loops, or even standalone blocks) have block scope.

- These variables are only accessible within the specific block where they are declared.

```
let globalVar = "I am global";            // Global scope

function exampleFunction() {
  var functionVar = "I am function-scoped";          // Function scope (with var)

  if (true) {
    let blockVar = "I am block-scoped";                    // Block scope (with let)
    console.log(blockVar);                                 // Accessible here
  }
```

3. **What problem does asynchronous JavaScript solve in single-threaded execution?**
   Answer:
   Asynchronous JavaScript is a programming approach that enables the non-blocking execution of tasks, allowing concurrent operations, improved responsiveness, and efficient handling of time- consuming operations in web applications.

4. **How the 'this' Keyword will process in JavaScript Programming explain** with an example?

Answer:
The 'this' keyword refers to the object that is currently executing or calling the function. It is a way to access and manipulate the properties and methods of the object that is called the function.

Example:

```
const person = {
name: "abc",
showName() {
console.log(`Hello, my name is ${this.name}`);
}
};
person.showName();
```

output:
Hello, my name is abc

5. **Explain the Types of Destructuring with Syntax in JavaScript Programming with simple Examples?**
   Answer:
   - **Destructuring** is a JavaScript syntax that extracts values from arrays or objects into distinct variables.
   - **Array Destructuring:** Array destructuring uses square brackets [] on the left-hand side of the assignment operator to unpack values based on their position (index) in the array
   - **Object Destructuring:** Object destructuring uses curly braces {} on the left-hand side to unpack properties into variables with the samename as the property key.

   Destructuring in Function Parameters: Function parameter destructuring in JavaScript is a concise syntax that allows you to unwrap values from an object or an array passed as an argument directly into distinct variables within the function's parameter list.

   Nested Destructuring: Nested object destructuring in JavaScript allows you to extract properties from objects that are themselves properties of another object. To destructure a nested object, use nested curly braces that match the structure of the object you are accessing.

6. **Explain briefly about Object Destructuring in JavaScript Programming with an Example?**
   Answer:

   JavaScript's **destructuring syntax** is a powerful expression that makes it possible to unpack values from arrays or properties from objects into distinct, local variables.
   Destructuring is a JavaScript syntax that extracts values from arrays or objects into distinct variables.

   Example:
   let student = {name:' abc,age:20};
   let {name, age} = student;
   console.log(name); // abc
   console.log(age); // 20

7. **What is the Syntax for Arrow Function in JavaScript Programing, Explain with an example?**
   Answer:

   Arrow Functions were introduced in ES6 and are commonly used in modern JavaScript.
- Arrow Functions allow a shorter syntax for function expressions. If thefunction body contains only one statement: You can remove the word function, the curly brackets and the return keyword.

Syntax:
An arrow function uses the **=>** symbol.
It is always written as a function expression.

Example:
```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Functions</h1>
<h2>The Arrow Function</h2>
<script>
// myFunction computes the product of a and b
const myFunction = (a, b) => a * b;
let result = myFunction(4, 5);
console.log("The product is:", result);
</script>
</body>
</html>
```

Output:
The product is: 20

Long Answer Questions

1. **Explain Default binding and Implicit binding of "this" keyword along with example program?**

Answer:

Default binding of "this" Keyword:
When a function is called in the global scope, without any object reference, the 'this' keyword refers to the global window object. This is called default binding.

Example:
```
function showThis() {
console.log(this);
}
showThis(); // Window object
```
output:
Window

Implicit binding of "this" Keyword:
In JavaScript, **implicit binding** is a rule that determines the value of the **"this" keyword based on the object a function** is called on at the time of invocation. This means this automatically refers to the object immediately to the left of the dot (.) when a method is called.

Example:

```
const person = {
name: "abc",
showName()
{
console.log("Hello, my name is" + this.name);
}
};
person.showName();
```

output:
Hello, my name is abc

2. **How is Error Handling will handle in JavaScript Programing? Explain the Error handling Mechanisms?**
Answer:

While writing JavaScript programs, it is very common to make mistakes. These mistakes are called **errors**. An error happens when JavaScript is unable to understand your code or cannot run it properly. When an error occurs, the program may stop suddenly, or it may behave in an unexpected way.

Common reasons for errors include:

- Typing mistakes while writing code
- Writing wrong logic
- Using a variable that has not been created
- Trying to use something before it exists
- Internet or server problems while fetching data

Types of Errors in JavaScript:

- **Syntax Error**: Happens when code grammar is wrong
- **Reference Error**: Happens when something does not exist
- **Type Error**: Happens when a value is used in the wrong way

Error handling is a technique used to **control errors instead of letting them crash the entire program**. Instead of stopping the program suddenly, error handling allows the program to respond in a proper and safe way.

Error handling in JavaScript is primarily managed using the **try...catch...finally** statement. This construct allows you to execute a block of code, catch and manage any errors that occur within it, and run a final block of code regardless of the outcome. The try...catch block is used to test a block of code for errors.

Syntax:

```
try {
.....
// Code that might throw an error
}
catch (error) {
......
// Code to handle the error
}
```

try block: Contains the code that is monitored for errors or will contain risky code.

catch block:
The catch block runs only if an error occurs inside the try block. It contains the code that runs when an exception is thrown in the try block.

It receives the error object as an argument, which contains information about the error. If no error happens, the catch block is skipped.

Finally statement:
The finally block executes after both the try and catch blocks have completed, regardless of whether an error was thrown or caught. It is often used for cleanup operations, like closing files or releasing resources.

Example:

```
<script>
try{
console.log(a);
}
catch(error)
{
console.log("something went wrong");
}
</script>
```

Output:

```
something went wrong                                    try-catch.html:12
```

3. **Explain briefly about Classes mechanism in JavaScript Programming in detail along with example?**

Answer:

A class Object defines the properties (data) and methods (functions) that all objects created from it will share. **In JavaScript, a class is a blueprint or template for creating objects that share similar properties and methods.**

class keyword: A class is declared using the class keyword, followed by the class name. Class names are typically capitalized by convention.

e.g., class Car { } .

constructor() method: A special method automatically called when a **new** object instance is created using the **new** keyword. It is used to initialize object properties.

new keyword: The mandatory keyword used to create a new instance (object) from a class.

The Constructor Method

The constructor method is a special method:

- It has to have the exact name "constructor"
- It is executed automatically when a new object is created
- It is used to initialize object properties

If you do not define a constructor method, JavaScript will add an empty constructor method.

Class Methods

Class methods are created with the same syntax as object methods.
Use the keyword class to create a class.
Always add a constructor() method.
Then add any number of methods.

Syntax:

```
class ClassName {
constructor() { ... }
method_1() { ... }
method_2() { ... }
method_3() { ... }
}
```

Example program:

```
class Car {
constructor(name, year) {
this.name = name;
this.year = year;
}
// method to return car details
getDetails() {
return this.name + " - " + this.year;   }  }
```

```
// Creating objects
const myCar1 = new Car("Ford", 2014);
const myCar2 = new Car("Audi", 2019);
// Displaying output in console
console.log("Car 1:", myCar1.getDetails());
console.log("Car 2:", myCar2.getDetails());
```

Output:
['Car 1:', 'Ford - 2014']
['Car 2:', 'Audi - 2019']

4. **Write the Syntax and Program of any ten Array Methods in JavaScript Programming?**
   Answer:

   JavaScript provides a rich set of built-in **array methods** to efficiently manipulate, iterate, search, and transform array data. JavaScript provides many **built-in array methods** to work with array elements easily.
   Examples:

1. push() – Add element at the end
 	
let arr = [1, 2, 3];
arr.push(4);
 	
console.log(arr);
	2. pop() – Remove last element
	
	let arr = [1, 2, 3];
	arr.pop();
	
	console.log(arr);
		3. unshift() – Add element at the beginning
		
		let arr = [2, 3];
		arr.unshift(1);
		
		console.log(arr);

// Output: [1, 2, 3, 4]	// Output: [1, 2]	// Output: [1, 2, 3]

4. shift() – Remove first element
 	
let arr = [1, 2, 3];
arr.shift(); console.log(arr);
 	
// Output: [2, 3]
	5. length – Get array size
	
	let arr = [10, 20, 30];
	
	console.log(arr.length);
	
	// Output: 3
		6. concat() – Merge arrays
		
		let a = [1, 2];
		let b = [3, 4];
		
		let result = a.concat(b);
		
		console.log(result);
		
		// Output: [1, 2, 3, 4]

7.slice() – Extract part of array
 	
let arr = [10, 20, 30, 40];
 	
let result = arr.slice(1, 3);
 	
console.log(result);
 	
// Output: [20, 30]
	8.splice()–Add/Remove elements
	
	let arr = [1, 2, 3, 4]; arr.splice(1, 2);
	
	console.log(arr);
	
	// Output: [1, 4]
		9. forEach() – Loop through array
		
		let arr = [1, 2, 3];
		
		arr.forEach(num => console.log(num));
		
		// Output:
		// 1
		// 2
		// 3

10. map() – Transform array
 	
let arr = [1, 2, 3]; let newArr = arr.map(num => num * 2);
 	
console.log(newArr);
 	
// Output: [2, 4, 6]
	11. sort() – Sort array
	
	let arr = [3, 1, 4, 2];
	
	arr.sort(); console.log(arr);
	
	// Output: [1, 2, 3, 4]
		12. reverse() – Reverse array
		
		let arr = [1, 2, 3];
		
		arr.reverse();
		
		// Output: [3,2,1]

14.toString() – Convert array to string
 	
let arr = [1, 2, 3];
 	
console.log(arr.toString());
 	
// Output: 1,2,3
	15.Array.isArray() – Check array
	
	let arr = [1, 2];
	
	console.log(Array.isArray(arr));
	
	// Output: true
		16.copyWithin() – Copy part inside array
		
		let arr = [1, 2, 3, 4];
		
		arr.copyWithin(1, 2); // we copy
		
		elements from index 2 to end
		
		console.log(arr);
		
		// Output: [1, 3, 4, 4]

5. **Write the Syntax and Program of any ten String Methods in JavaScript Programming?**

Answer:

JavaScript provides many built-in string methods to manipulate and work with text data. Strings are immutable, meaning these methods return a new string, leaving the original string unchanged.

String Methods and its Examples:

1. length – Get string length
 	
let str = "Hello";
 	
console.log(str.length);
 	
// Output: 5
	2. toUpperCase() – Convert to uppercase
	
	let str = "hello";
	
	console.log(str.toUpperCase());
	
	// Output: HELLO
		3. toLowerCase() – Convert to lowercase
		
		let str = "HELLO";
		
		console.log(str.toLowerCase());
		
		// Output: hello

4. trim() – Remove spaces (start & end) **let str = " Hello ";** **console.log(str.trim());** **// Output: Hello**	**5. charAt() – Get character at index** **let str = "Hello";** **console.log(str.charAt(1));** **// Output: e**	**6. indexOf() – Find position** **let str = "Hello World";** **console.log(str.indexOf("World"));** **// Output: 6**
7. lastIndexOf() – Last occurrence **let str = "Hello Hello";** **console.log(str.lastIndexOf("Hello"));** **// Output: 6**	**8. slice() – Extract part** **let str = "JavaScript";** **console.log(str.slice(4, 10));** **// Output: Script**	**9. substring() – Extract part** **let str = "JavaScript";** **console.log(str.substri ng(0, 4));** **// Output: Java**

10.replace() – Replace text **let str = "I like JS";** **console.log(str.replace(" JS", "JavaScript"));** **// Output: I like JavaScript**	**11. replaceAll() – Replace all matches** **let str = "cat cat dog";** **console.log(str.replaceAll("cat", "rat"));** **// Output: rat rat dog**	**12. includes() –** **Check existence** **let str = "Hello World";** **console.log(str.includes("World"));** **// Output: true**
13.startsWith() / endsWith() **let str = "JavaScript";** **console.log(str.startsWith("Java"));** **console.log(str.endsWit**	**14. concat() – Join strings** **let a = "Hello"; let b = "World";** **console.log(a.concat(" ", b));**	**15. repeat() – Repeat string** **let str = "Hi ";** **console.log(str.repeat(3));**

6. **Explain in detail about try block, catch block and finally block in JavaScript Programming along with example program?**

Answer:

While writing JavaScript programs, it is very common to make mistakes. These mistakes are called **errors**. An error happens when JavaScript is unable to understand your code or cannot run it properly. When an error occurs, the program may stop suddenly, or it may behave in an unexpected way.

Common reasons for errors include:

- Typing mistakes while writing code
- Writing wrong logic
- Using a variable that has not been created
- Trying to use something before it exists
- Internet or server problems while fetching data

Types of Errors in JavaScript:

- **Syntax Error**: Happens when code grammar is wrong
- **Reference Error**: Happens when something does not exist
- **Type Error**: Happens when a value is used in the wrong way

Error handling is a technique used to **control errors instead of letting them crash the entire program**. Instead of stopping the program suddenly, error handling allows the program to respond in a proper and safe way.

Error handling in JavaScript is primarily managed using the **try...catch...finally** statement. This construct allows you to execute a block of code, catch and manage any errors that occur within it, and run a final block of code regardless of the outcome.
The try...catch block is used to test a block of code for errors.
Syntax:

```
try {
      .....
  // Code that might throw an error

}
 catch (error) {
      ......
  // Code to handle the error
}
```

try block: Contains the code that is monitored for errors or will contain risky code.
catch block:
The catch block runs only if an error occurs inside the try block. It contains the code that runs when an exception is thrown in
the try block.
It receives the error object as an argument, which contains information about the error. If no error happens, the catch block is skipped.

Finally statement:
The finally block executes after both the try and catch blocks have completed, regardless of whether an error was thrown or caught. It is often used for cleanup operations, like closing files or releasing resources.

Example:

```
<script>
try{
console.log(a);
}
catch(error)
{
console.log("something went wrong");
}
</script>
```

Output:

```
something went wrong                                    try-catch.html:12
```

MODULE-II
Short Answer Questions

1. **What problem does asynchronous JavaScript solve in single-threaded execution?**

Answer:

- **Non-blocking:** The program continues executing subsequent lines of code while waiting for an asynchronous task to finish.
- **Single-threaded:** JavaScript is naturally single-threaded, meaning it executes one command at a time. It achieves concurrency through its non-blocking behaviour and the event loop.
- **Event Loop:** This mechanism manages the execution of multiple tasks by moving completed asynchronous callbacks from a queue into the call stack once it is empty.

➢ There are mainly two types in the Asynchronous programming :
- Callback functions
- Callback Hell and management

2. **How Synchronous programming will process in JavaScript code Explain in detail ?**

Answer:

- **Synchronous programming** refers to a coding execution model where operations are performed one after the other in a strict, linear sequence. Each statement must be completed and an answer returned before the next one can begin.
- By default, JavaScript is a synchronous programming, single-threaded language.

Example:

```
console.log("First task");
console.log("Second task");
console.log("Third task");
```

output:

```
First task
Second task
Third task
```

3. **Explain the Promise states in JavaScript Programming of following**
   a) **Pending state**
   b) **Fulfilled state**
   c) **Reject state**

Answer:

- Promises have three states:
  - ***Pending****: The initial state. The asynchronous operation has not yet completed. The result is initially undefined.*
  - ***Fulfilled (or Resolved)****: The operation completed successfully, and a resulting value is available.*
  - ***Rejected****: The operation failed, and an error object (or "reason") is available.*

Structure of a Promise

The basic syntax for creating a promise is:

```
const myPromise = new Promise((resolve, reject) => {
  // Asynchronous operation
  here if (/* operation
  successful */) {
    resolve('Success');
  } else {
    reject('Erro
    r');
}
});
```

4. **Explain the methods that can resolve in promise of JavaScript Programing?**

Answer:

.then(): .then() is used to handle a fulfilled (resolved) Promise. Runs only when resolve() is called.
Syntax: promise.then(successCallback);

.catch(): .catch() is used to handle a rejected Promise. Runs only when reject() is called.
Syntax: promise.catch(errorCallback);

For example:

```
myPromise
.then(result => console.log(result))
.catch(error => console.log(error));
```

5. **Explain briefly about states in promise Asynchronous operation in JavaScript Programming?**
   Answer:

   Promises have three states:

1. **Pending**: The initial state. The operation hasn't finished yet, and the promise is neither fulfilled nor rejected.
2. **Fulfilled (Resolved)**: The operation completed **successfully**. The promise now has a resulting value (e.g., data from an API).
3. **Rejected**: The operation **failed**. The promise now has a reason for the failure (usually an error object).

   Structure of a Promise
   The basic syntax for creating a promise is:

```
const myPromise = new Promise((resolve, reject) => {
// Asynchronous operation here if (/* operation successful */) {
resolve('Success');
} else {

reject('Error');
}
});
```

6. **How Asynchronous programming will process in JavaScript code Explain in detail ?**

Answer:

Asynchronous programming in JavaScript allows the program to start long-running tasks (like network requests or data fetching) and continue executing other code without waiting for the task to finish, thereby preventing the application from freezing.

The main mechanisms for handling asynchronous operations are callbacks, Promises, and the modern async/await syntax.

1. Callbacks
A callback is a function passed as an argument to another function, which is then executed when the asynchronous operation completes. This was the original way to handle async operations.

2. Promises
Promises provide a cleaner, more structured approach than callbacks (which can lead to "callback hell" with deep nesting). A promise represents the eventual result of an asynchronous operation and can be in one of three states: pending, fulfilled, or rejected.

Long Answer Questions

1. **What is a Callback Function, and how does it enable asynchronous behaviour in JavaScript explain in detail with an Example?**
   Answer:

   A callback function is a function that is passed as an argument to another function and is executed after the completion of some operations. This mechanism allows JavaScript to perform tasks like reading files, making HTTP requests, or waiting for user input without blocking the execution of the program. This helps ensure a smooth user experience.

   Why Use Callback Functions?
   JavaScript runs in a single-threaded environment, meaning it can only execute one command at a time. Callback functions help manage asynchronous operations, ensuring that the code continues to run smoothly without waiting for tasks to complete. This approach is crucial for maintaining a responsive and efficient program.

   - A function can accept another function as a parameter.
   - Callbacks allow one function to call another at a later time.
   - A callback function can execute after another function has finished.

   Syntax:

   function outerFunction(data, callback)
   {
   // Perform some task
   callback(data); // Executing the callback
   }

   Example:

```
function greet(name, callback) {
  console.log("Hello " + name);  'Hello Husna'
  callback();
}

function sayBye() {
  console.log("Goodbye!");  'Goodbye!'
}

greet("Husna", sayBye);
```

```
LOGS
> 'Hello Husna'
> 'Goodbye!'
```

   Why Do We Use Callback Functions?
   JavaScript is **asynchronous** (non-blocking). Callbacks help us:
   - Handle **delays** (API calls, timers)
   - Control **execution order**
   - Run code **after a task finishes.**

2. **Demonstrate how to implement time delays in JavaScript programming with an example?**
   Answer:

   JavaScript provides two primary functions, setTimeout() and setInterval(), to schedule the execution of code asynchronously.
   setTimeout(): The setTimeout() method executes a function only once after a specified delay in milliseconds.

   Syntax:let timeoutID = setTimeout(function, delay, [arg1, arg2, ...]);

   - Parameters:
   - function: The function to be executed.
   - delay: The time, in milliseconds (1000ms = 1 second), to wait before the function is called. The default is 0.
   - arg1, arg2, ... (Optional): Additional arguments passed directly to the function when it is executed.

   Example:

```
function step1() {
  console.log(" execute step 1 ")  ' execute step 1 '
}

function step2() {
  console.log(" execute step 2 ")  ' execute step 2 '
}

setTimeout(step1,3000);
setTimeout(step2,1000);
```

```
LOGS
> ' execute step 2 '
> ' execute step 1 '
```

   setInterval(): The setInterval() method in JavaScript is used to repeatedly execute a specified function or code at fixed time intervals. It continues running until explicitly stopped using clearInterval() or the browser window is closed.

   Syntax: let intervalID = setInterval(function, delay, [arg1, arg2, ...]);

   - function (required): The function to be executed repeatedly.
   - delay (required): The time interval between each execution, specified in milliseconds (1000 milliseconds = 1 second).
   - arg1, arg2, ... (optional): Additional arguments that are passed to the executed function.

   - **intervalID:** The method returns a unique numeric ID that identifies the created timer. This ID is essential for stopping the interval later with clearInterval().
     Note: 1 second = 1000 milliseconds.
     By using setInterval() method, the output will be printed after 1 sec or 1000ms indefinitely unless you don't stop it or close window.

Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using clearInterval() to stop the execution of the setInterval().

```
function myFunction() {
    console.log("Executing at regular intervals!");  'Execut
}

// store interval id
let intervalId = setInterval(myFunction, 1000);

// stop the interval after 5 seconds
setTimeout(() => {
    clearInterval(intervalId);
    console.log("Interval stopped!");  'Interval stopped!'
}, 5000);
```

Output:

```
LOGS
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Interval stopped!'
```

In this example:

- setInterval() returns an ID → intervalId
- clearInterval(intervalId) uses that ID to stop execution
- setTimeout() is used to decide *when* to stop it (after 5 seconds)
- clearInterval() only works with the **interval ID**, not the function name.

3. **How does overusing callbacks lead to unmaintainable code, and what are the solutions Explain in detail?**
   Answer:

   "Callback hell," also known as the "Pyramid of Doom," is a problem in JavaScript that occurs when multiple asynchronous operations are deeply nested using callback functions. This results in code that is difficult to read, debug, and maintain.

   What is Callback Hell?
   Callback Hell is a situation where multiple nested callbacks make the code difficult to read and maintain.
   In JavaScript, callbacks are functions passed as arguments to other

functions, to be executed later when an asynchronous operation (like fetching data, reading a file, or a timer) completes.

When multiple asynchronous tasks depend on the result of the previous one, developers historically nested the subsequent callbacks inside the previous ones. This creates a deeply indented, pyramid-like code structure that grows horizontally, making the program's flow difficult to follow.

Callback Hell happens when:

- You have many nested callbacks
- Code becomes hard to read, hard to debug, and hard to maintain

It looks like a pyramid / triangle of functions.

Example:

```
setTimeout(() => {
  console.log("Step 1");  'Step 1'

  setTimeout(() => {
    console.log("Step 2");  'Step 2'

    setTimeout(() => {
      console.log("Step 3");  'Step 3'

      setTimeout(() => {
        console.log("Step 4");  'Step 4'
      }, 1000);

    }, 1000);

  }, 1000);

}, 1000);
```

```
LOGS
> 'Step 1'
> 'Step 2'
> 'Step 3'
> 'Step 4'
```

As seen above, each function is nested within the one before it, forming the characteristic "pyramid" shape.

Why is it a problem?

- Poor Readability: The increasing indentation and fragmented logic make the code very hard to read and understand.
- Difficult Maintenance: Modifying or adding new features to such a structure is error-prone and time-consuming.
- Complex Error Handling: Error management becomes cumbersome, as each nested level must handle potential errors independently, leading to repetitive boilerplate code.
- Debugging Challenges: Tracing the flow of execution and locating bugs within deeply nested functions can be a nightmare.

How to AVOID Callback Hell:

Modern JavaScript (ES6+ and Node.js) provides much cleaner ways to handle asynchronous operations and avoid this issue.

- Promises: Promises represent the eventual result of an asynchronous operation and allow you to chain operations

using .then() and .catch() methods, which flatten the code structure.

- async/await: Built on top of Promises, async/await provides a syntax that allows asynchronous code to be written as if it were synchronous, making it the most readable and manageable solution.

- **Modularization/Named Functions:** Breaking the code into smaller, reusable, named functions can reduce nesting even when using the callback pattern, making the code more organized.

Example: Use Named Functions

```
function step1() {
  console.log("Step 1");  'Step 1'
  setTimeout(step2, 1000);
}

function step2() {
  console.log("Step 2");  'Step 2'
  setTimeout(step3, 1000);
}

function step3() {
  console.log("Step 3");  'Step 3'
}

setTimeout(step1, 1000);
```

```
LOGS
> 'Step 1'
> 'Step 2'
> 'Step 3'
```

4. **How do JavaScript timing events (setTimeout and setInterval) work, what are their differences, and Explain their implementation with examples?**
   Answer:

   JavaScript provides two primary functions, setTimeout() and setInterval(), to schedule the execution of code asynchronously.
   setTimeout(): The setTimeout() method executes a function only once after a specified delay in milliseconds.

   Syntax:let timeoutID = setTimeout(function, delay, [arg1, arg2, ...]);
   - Parameters:
   - function: The function to be executed.
   - delay: The time, in milliseconds (1000ms = 1 second), to wait before the function is called. The default is 0.
   - arg1, arg2, ... (Optional): Additional arguments passed directly to the function when it is executed.

   Example:

```
function step1() {
  console.log(" execute step 1 ")  ' execute step 1 '
}

function step2() {
  console.log(" execute step 2 ")  ' execute step 2 '
}

setTimeout(step1,3000);
setTimeout(step2,1000);
```

```
LOGS
> ' execute step 2 '
> ' execute step 1 '
```

setInterval(): The setInterval() method in JavaScript is used to repeatedly execute a specified function or code at fixed time intervals. It continues running until explicitly stopped using clearInterval() or the browser window is closed.

Syntax: let intervalID = setInterval(function, delay, [arg1, arg2, ...]);

- function (required): The function to be executed repeatedly.
- delay (required): The time interval between each execution, specified in milliseconds (1000 milliseconds = 1 second).
- arg1, arg2, ... (optional): Additional arguments that are passed to the executed function.
- intervalID: The method returns a unique numeric ID that identifies the created timer. This ID is essential for stopping the interval later with clearInterval().

Note: 1 second = 1000 milliseconds.

By using setInterval() method, the output will be printed after 1 sec or 1000ms indefinitely unless you don't stop it or close window.

Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using clearInterval() to stop the execution of the setInterval().

```
function myFunction() {
    console.log("Executing at regular intervals!");  'Execut
}

// store interval id
let intervalId = setInterval(myFunction, 1000);

// stop the interval after 5 seconds
setTimeout(() => {
    clearInterval(intervalId);
    console.log("Interval stopped!");  'Interval stopped!'
}, 5000);
```

Output:

```
LOGS
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Executing at regular intervals!'
> 'Interval stopped!'
```

In this example:

- setInterval() returns an ID → intervalId
- clearInterval(intervalId) uses that ID to stop execution
- setTimeout() is used to decide *when* to stop it (after 5 seconds)
- clearInterval() only works with the **interval ID**, not the function name.

5. Explain the concept of "Pyramid of Doom" in detail along with example program?

Answer:

"Callback hell," also known as the "Pyramid of Doom," is a problem in JavaScript that occurs when multiple asynchronous operations are deeply nested using callback functions. This results in code that is difficult to read, debug, and maintain.

What is Callback Hell?
Callback Hell is a situation where multiple nested callbacks make the code difficult to read and maintain.
In JavaScript, callbacks are functions passed as arguments to other functions, to be executed later when an asynchronous operation (like fetching data, reading a file, or a timer) completes.

When multiple asynchronous tasks depend on the result of the previous one, developers historically nested the subsequent callbacks inside the previous ones. This creates a deeply indented, pyramid-like code structure that grows horizontally, making the program's flow difficult to follow.

Callback Hell happens when:

- You have **many nested callbacks**
- Code becomes **hard to read, hard to debug, and hard to maintain**

It looks like a **pyramid / triangle** of functions.

Example:

```
setTimeout(() => {
  console.log("Step 1");  'Step 1'

  setTimeout(() => {
    console.log("Step 2");  'Step 2'

    setTimeout(() => {
      console.log("Step 3");  'Step 3'

      setTimeout(() => {
        console.log("Step 4");  'Step 4'
      }, 1000);

    }, 1000);

  }, 1000);

}, 1000);
```

```
LOGS
> 'Step 1'
> 'Step 2'
> 'Step 3'
> 'Step 4'
```

As seen above, each function is nested within the one before it, forming the characteristic "pyramid" shape.

Why is it a problem?

•Poor Readability: The increasing indentation and fragmented logic make the code very hard to read and understand.

•Difficult Maintenance: Modifying or adding new features to such a structure is error-prone and time-consuming.

•Complex Error Handling: Error management becomes cumbersome, as each nested level must handle potential errors independently, leading to repetitive boilerplate code.

•Debugging Challenges: Tracing the flow of execution and locating bugs within deeply nested functions can be a nightmare.

How to AVOID Callback Hell:

Modern JavaScript (ES6+ and Node.js) provides much cleaner ways to handle asynchronous operations and avoid this issue.

•Promises: Promises represent the eventual result of an asynchronous operation and allow you to chain operations
using .then() and .catch() methods, which flatten the code structure.

•async/await: Built on top of Promises, async/await provides a syntax that allows asynchronous code to be written as if it were synchronous, making it the most readable and manageable solution.

•Modularization/Named Functions: Breaking the code into smaller, reusable, named functions can reduce nesting even when using the callback pattern, making the code more organized.

Example: Use Named Functions

```
function step1() {
  console.log("Step 1");   'Step 1'
  setTimeout(step2, 1000);
}

function step2() {
  console.log("Step 2");   'Step 2'
  setTimeout(step3, 1000);
}

function step3() {
  console.log("Step 3");   'Step 3'
}

setTimeout(step1, 1000);
```

```
LOGS
> 'Step 1'
> 'Step 2'
> 'Step 3'
```

6. **How do Promises manage asynchronous operations, and what is their constructor structure and Explain briefly with an example?**

Answer:

A Promise is a constructor function in JavaScript used to create Promise objects for handling asynchronous operations in JavaScript represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

A Promise is in one of these states:

- Pending: The initial state. The asynchronous operation has not yet completed. The result is initially undefined.
- Fulfilled (or Resolved): The operation completed successfully, and a resulting value is available.
- Rejected: The operation failed, and an error object (or "reason") is available.

Structure of a Promise

The basic syntax for creating a promise is:

```
const myPromise = new Promise((resolve, reject) => {
// Asynchronous operation here if (/* operation successful */) {
resolve('Success');
} else { reject('Error');
}
});
```

Creating Promises

A Promise is created using the Promise constructor with the new keyword.

Basic syntax

```
const promise = new Promise((resolve, reject) => {
// asynchronous task
});
```

Example:

```
const myPromise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Task completed successfully");
    } else {
        reject("Task failed");
    }
});
myPromise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

Output:

```
LOGS
> Task completed successfully
```

How it works

- Promise is **created**
- success = true → resolve() is called
- .then() receives the value
- Output is printed on the console.

MODULE III
Short Answers

1. How does JSX bridge the gap between HTML markup and JavaScript logic in React?

Ans:

It allows developers to write familiar HTML-like markup directly within JavaScript code, which enhances code readability and simplifies UI development.

Core Concepts and Usage

- **HTML-like Syntax:** JSX looks very similar to HTML, using tags and attributes to define the UI structure within the same file as the rendering logic.
- **Transpilation:** Browsers cannot read JSX directly. A transpiler like Babel converts the JSX code into standard JavaScript function calls, specifically React.createElement() calls, which the browser can understand.
- **React Elements:** JSX expressions are essentially syntactic sugar that evaluate to plain JavaScript objects called "React elements."
- **Component Structure:** In React, components are functions or classes that return JSX, enabling a modular, component-based approach to building UIs.

2. Explain the core concepts of React and its significance in modern front-end development?

Ans:

React is a JavaScript library used to build user interfaces (UI).

It was developed by Facebook (Meta) and is widely used for building fast, interactive web apps.

React focuses only on the view layer (UI), not the entire application.

1. **JSX (JavaScript XML):** This allows you to write HTML-like code directly inside your JavaScript. It makes the structure of your UI much easier to visualize.
2. **The Virtual DOM:** Normally, updating a webpage is slow. React creates a "virtual" copy of the page in its memory, figures out exactly what changed, and only updates that specific part.

3. **Unidirectional Data Flow:** Data in React flows in one direction (downwards from parents to children). This makes debugging much simpler.
4. **State and Props:**
    - props are used to pass data from a parent to a child
    - while state is managed internally within a component for data that changes over time.

3. What role does JSX play in React development, and how does it simplify component creation?
Ans:
JSX (JavaScript XML) is a syntax extension for JavaScript recommended for use with React to describe what the user interface (UI) should look like. It allows developers to write familiar HTML-like markup directly within JavaScript code, which enhances code readability and simplifies UI development.

In React, components are functions or classes that return JSX, enabling a modular, component-based approach to building UIs.

- A component must return a single top-level element. To group multiple elements, wrap them in a or a <>...</> (Fragment) to avoid adding unnecessary nodes to the DOM.

- All tags, including typically self-closing HTML elements like <img /> and <input />, must be explicitly closed with a trailing slash (e.g., <img src="..." />).

- You can embed any valid JavaScript **expression** (variables, function calls, arithmetic operations, etc.) within JSX by wrapping it in curly braces { }.

Example: const name = 'Sara'; <h1>Hello, {name}</h1>.

4. Explain the role of the Virtual DOM in updating user interfaces efficiently?
Ans:
Virtual DOM is a lightweight copy of the Real DOM kept in memory.
In React, the Virtual DOM (VDOM) makes the UI efficient. Instead of touching the actual browser DOM every time something changes—which is slow—React uses a lightweight, memory-resident copy.

How the Virtual DOM Works
The process involves three main steps whenever a component's state or props change:

1. **Creation**: A new Virtual DOM tree is created that represents the updated UI state.
2. **Diffing**: React uses a "diffing" algorithm to compare the new Virtual DOM tree with the previous snapshot of the Virtual DOM tree.

- The Flow is
  Like This:

State changes
↓
Create NEW Virtual DOM
↓
Compare with OLD Virtual DOM (Diffing happens here)
↓
Find differences
↓
Update Real DOM only where changed

3. **Reconciliation (Updating the Real DOM)**: After identifying the minimal changes, React applies only those specific changes to the actual DOM. This is a batch update, meaning multiple updates are grouped and performed in a single pass, which significantly reduces the time and resources spent on layout recalculations and repaints by the browser.

5. What are the differences between functional components and class components in React?
Ans:

Functional components:
In React, components are the building blocks of a user interface. They are reusable, self- contained pieces of code that represent a part of the UI. React allows you to break down your UI into smaller components, which makes it easier to manage and maintain your codebase.
Class Components:
It was the traditional way of writing components before the introduction of Hooks in React. Today, most developers prefer Function Components with Hooks, but understanding Class Components is still important.
When creating a React component, the component name must start with an uppercase letter.

A class component must also include a render() method. This method returns JSX (HTML-like code) that will be displayed on the screen

6. Explain the role of props in React and why a child component cannot change its own props?
Ans: Props (short for properties) are used in React to pass data from one component to another. They allow components to communicate with each other and make components reusable.
What are Props?
Props are read-only inputs passed from a parent component to a child component.
They help in:
- Passing data
- Passing functions
- Making components reusable
- Maintaining unidirectional data flow

Key Points

- Props are immutable (cannot be changed) by the child component.
- Props are passed from parent → child.
- Props help components become dynamic and reusable.

Long Answers:

1. What are the core principles and features of the React library Explain with an Example?

Ans:

React is a JavaScript library used to build user interfaces (UI).
it was developed by Facebook (Meta) and is widely used for building fast, interactive web apps.
React focuses only on the view layer (UI), not the entire application.
React stays popular because it solves specific problems that used to make web development as complex:

- **JSX (JavaScript XML):** This allows you to write HTML-like code directly inside your JavaScript. It makes the structure of your UI much easier to visualize.
- **The Virtual DOM:** Normally, updating a webpage is slow. React creates a "virtual" copy of the page in its memory, figures out exactly what changed, and only updates that specific part.
- **Unidirectional Data Flow:** Data in React flows in one direction (downwards from parents to children). This makes debugging much simpler.
- **Props**

props are used to pass data from a parent to a child.
Props act like function arguments, allowing parent components to pass data to child components and customize their appearance.
Unidirectional Flow: Data with props flows from parent to child components in a one-way direction.
Read-Only (Immutable): A component cannot change its own props. They are read-only to the receiving component.
Reusable Components: Props make components reusable. The same child component can render different data based on the props it receives from its parent.
Communication: They are primarily used for communication between components, specifically from a parent to a child.

Example:

Propsexample.jsx	**App.js**

function Propsexample(props)
{
return(
 	
<h1>Name:{props.name}</h1>
<h1>Htno:{props.htno}</h1>
 	
)
}
	import Propsexample from
	'./Propsexample';
	function App() {
	
	return (
	
	<Propsexample name='abc'
	htno={123}/>
	
	);

export default Propsexample;	} export default App;

- **State**

State is like a component's memory or internal data storage. It is used to keep track of information that can change over time, typically in response to user interactions, network responses, or other events.
Local and Encapsulated: State is local to the component where it is defined and cannot be accessed or modified directly by other components.
Mutable: The state can be updated within the component using specific methods: useState hook in functional components or this.setState in class components.
Triggers Re-render: When the state of a component changes, React automatically re-renders that component (and its children) to reflect the new data.
Dynamic UIs: State is essential for creating dynamic and interactive user interfaces, such as toggles, counters, and form inputs

Useeffect.jsx	**App.js**

import { useEffect, useState } from "react";
function Useeffect()
{
const [timer,setTimer]=useState(0)
useEffect(()=>
console.log('inside use effect'));
return (
<h1>Timer:{timer};
{console.log('inside componenet')}
 	
<button onClick={()=>{setTimer(timer+1)}}>Increment</button>
<button onClick={()=>{setTimer(timer-1)}}>Decrement</button>
</h1>
);
}
export default Useeffect;
	import Useeffect from './USeeffect';
	
	function App() {
	
	return (
	
	< Useeffect />
	
	);
	}
	
	export default App;

2. **What are the different ways to create components in React and when should I use them Explain with a suitable Example?**

   Ans:

   In React, components are the building blocks of a user interface. They are reusable, self-contained pieces of code that represent a part of the UI. React allows you to break down your UI into smaller components, which makes it easier to manage and maintain your codebase.
   You can think of components as custom HTML elements that encapsulate their own logic and UI structure.

There are two main types of components in React:

- A Functional Component in React is a simple JavaScript function that returns JSX (UI).
- A Class Component in React is a component created using a JavaScript class.

- Functional Components may create Components by following ways:

1. **Basic Syntax**

```
function Component_name(){
returns <h1> some heading </h1>
}
export default Component_name;
```

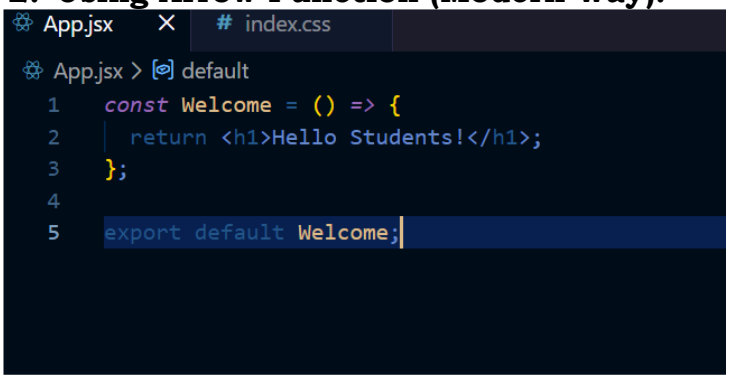
Example:

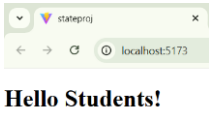
```
function Welcome() {
return <h1>Hello Students!</h1>;
}
export default Welcome;
```

Output:

Hello Students!

2. **Using Arrow Function (Modern Way):**





3. **Functional Component with Props:**

Functional components are simple JavaScript functions that accept props as an argument and return React elements (JSX) to define the UI.

Props = Properties

Props are used to **pass data from one component to another**.

Propsexample.jsx	**App.js**

function Propsexample(props)
{
return(
 	
<h1>Name:{props.name}</h1>
<h1>Htno:{props.htno}</h1>
 	
)
}
 	
export default Propsexample;
	import Propsexample from
	'./Propsexample';
	function App() {
	
	return (
	
	<Propsexample name='abc'
	htno={123}/>
	
	);
	}

	export default App;

4. **Hooks in Functional Component:**
   In modern React, Hooks are used only inside Functional Components, Class components do NOT use hooks.
   Here in below example:
   - useState is a Hook
   - It allows functional component to store data
   - When state changes → component re-renders

Useeffect.jsx	**App.js**

import { useEffect, useState } from "react";
function Useeffect()
{
const [timer,setTimer]=useState(0)
useEffect(()=>
console.log('inside use effect'));
return (
<h1>Timer:{timer};
{console.log('inside componenet')}
 	
</br>
<button onClick={()={setTimer(timer+1)}}>Increment</button>
<button onClick={()={setTimer(timer-1)}}>Decrement</button>
</h1>
);
}
export default Useeffect;
	import Useeffect from './USeeffect';
	
	function App() {
	
	return (
	
	< Useeffect />
	
	);
	}
	
	export default App;

- A Class Component in React is a component created using a JavaScript class.
  1. **Basic Structure of Class Component**

```
import React, { Component } from "react";
class Welcome extends Component
{
render()
 {
return <h1>Hello Students!</h1>;
 }
}
export default Welcome;
```

Here:

- class Welcome extends Component → Creating a class component
- extends Component → Inheriting React features
- render() → Must return JSX
- export default Welcome → Makes component usable in other files

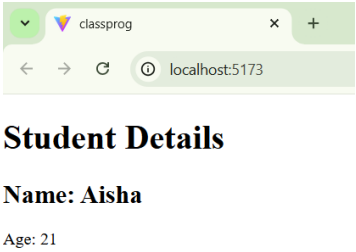
2. **Class Component with Props:**

```jsx
import React, { Component } from "react";

class Student extends Component {
  render() {
    return (
      <div>
        <h2>Name: {this.props.name}</h2>
        <p>Age: {this.props.age}</p>
      </div>
    );
  }
}

class App extends Component {
  render() {
    return (
      <div>
        <h1>Student Details</h1>
        <Student name="Aisha" age={21} />
      </div>
    );
  }
}

export default App;
```



1. **Class Component with State:**

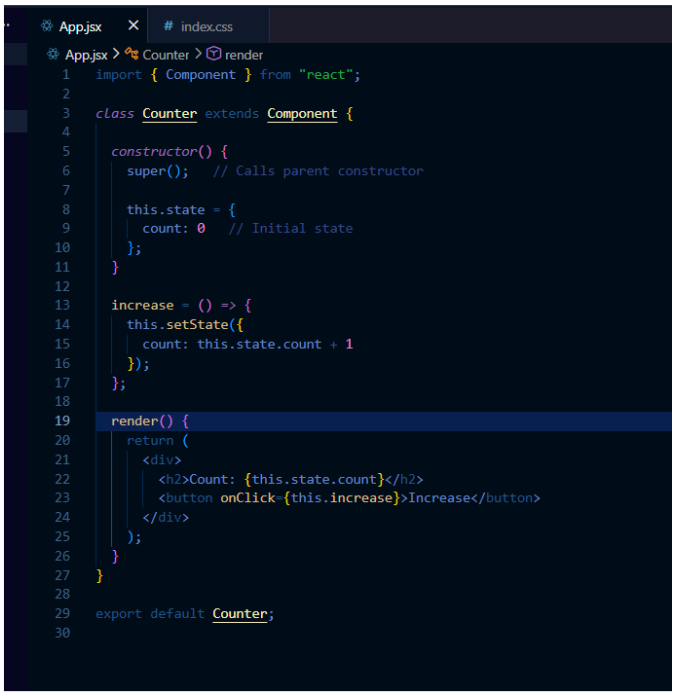
State is a variable that belongs to the component When state changes → Component re-renders (updates UI)
In class components:

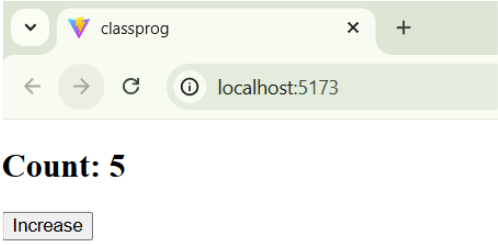
- We use this.state
- We update using this.setState()

this.state: this.state is the memory of a class component. It stores data that belongs to that component.
Example:
this.state = { count: 0 };
This means:

- The component remembers count
- Initial value is 0

this.setState(): this.setState() is used to **change the state value**When state changes:React updates the screen automatically.





3. Compare functional components and class components in React with examples.

Ans:

Functional Component in React

A Functional Component in React is a simple JavaScript function that returns JSX (UI). It is the modern and most commonly used way to create components in React.

Basic Syntax

function Component_name(){
returns <h1> some heading
</h1>
}
export default Component_name;

Example:

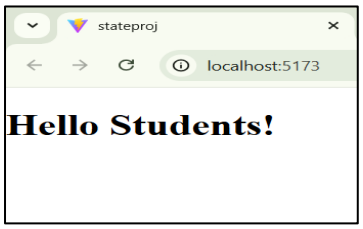
function Welcome() {
return <h1>Hello Students!</h1>;
}
export default Welcome;

Output:

Hello Students!

- function Welcome() → Component name (must start with CAPITAL letter)
- return → Returns JSX (HTML-like code)
- export default → Allows this component to be used in other files.

Using Arrow Function (Modern Way):



Both methods are correct.

Class Components

A Class Component in React is a component created using a JavaScript class. It was the traditional way of writing components before the introduction of Hooks in React. Today, most developers prefer Function Components with Hooks, but understanding Class Components is still important.

When creating a React component, the component name must start with an uppercase letter. A class component must include the statement extends React.Component. This allows the class to inherit features from React.Component.
A class component must also include a render() method. This method returns JSX (HTML-like code) that will be displayed on the screen.

Basic Structure of Class Component

```
import React, { Component } from
"react"; class Welcome extends
Component { render() {
   return <h1>Hello Students!</h1>;
  }
}
export default
Welcome; Here:
```

a. class Welcome extends Component → Creating a class component
b. extends Component → Inheriting React features
c. render() → Must return JSX
d. export default Welcome → Makes component usable in other files

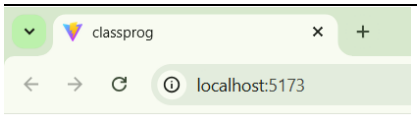
Class Component with Props:

```jsx
import React, { Component } from "react";

class Student extends Component {
  render() {
    return (
      <div>
        <h2>Name: {this.props.name}</h2>
        <p>Age: {this.props.age}</p>
      </div>
    );
  }
}

class App extends Component {
  render() {
    return (
      <div>
        <h1>Student Details</h1>
        <Student name="Aisha" age={21} />
      </div>
    );
  }
}

export default App;
```



Student Details

Name: Aisha

Age: 21

4. **Explain the Component Lifecycle in detail, covering the Mounting, Updating, and Unmounting phases.**

Ans:

In React, every class component goes through a lifecycle from creation to removal from the DOM. This lifecycle is divided into three main phases:

1. **Mounting Phase** – Component is created and inserted into the DOM
2. **Updating Phase** – Component updates when state or props change
3. **Unmounting Phase** – Component is removed from the DOM

Mounting Phase

In React, the Mounting Phase is the first stage of a component's lifecycle.

When a component is created and displayed on the screen for the first time.

When a React component loads, React performs some steps to prepare the component and show it in the browser (DOM).

Steps in Mounting Phase

There are 3 main methods executed in this phase.

1. **constructor():**

- This is the first method that runs when the component is created.
- It is used to initialize state and variables.

Example

```
constructor(props){
super(props);
 this.state = { count: 0 };
}
```

Simple idea: Constructor prepares the component.

2. **render():**
   - This method creates the UI.
   - It returns JSX (what should appear on the screen).

Example

```
render()
{
 return <h1>Hello Students</h1>;
}
```

Simple idea: Render tells React what to display.

3. **componentDidMount():**
   - This method runs after the component appears in the browser.
   - It is used for:
     - API calls
     - fetching data
     - starting timers

Example

```
componentDidMount(){
 console.log("Component
 Mounted");
}
```

Simple idea: This runs after the component is visible on the screen.

Order of Execution

constructor()

↓

render()

↓

componentDidMount()

Updating Phase

The Updating Phase happens when a component changes and re-renders. This occurs when:

- State changes
- Props change

React updates the UI automatically.

Example: If a counter value increases, the component updates and shows the new value.

Lifecycle Methods in Updating Phase

- shouldComponentUpdate()
- React checks whether the component should update or not.
- It returns true or false.

Example

```
shouldComponentUpdate()
{ return true;
}
```

Simple idea: React asks → Should I update the component?

2. render()
- React recreates the UI with updated data.

Example

```
render(){
  return <h1>{this.state.count}</h1>;
}
```

Simple idea:Render updates what appears on the screen.

1. componentDidUpdate()
- This method runs after the component updates.

Example

```
componentDidUpdate(){
  console.log("Component
  Updated");
}
```

Simple idea: Runs **after the screen**

changes. **Order of Updating Phase**

State/Props Change

↓

shouldComponentUpdate

()

↓

render()

↓

componentDidUpdate()

Unmounting Phase

The Unmounting Phase happens when the component is removed from the screen (DOM).

Example:

- Closing a page
- Removing a component
- Navigating to another page

Lifecycle Method

componentWillUnmount(): This method runs just before the component is removed.

Example

```
componentWillUnmount(){
  console.log("Component
  Removed");
}
```

Why it is used

It is used to **clean up resources** such as:

- Stop timers
- Cancel API calls
- Remove event listeners

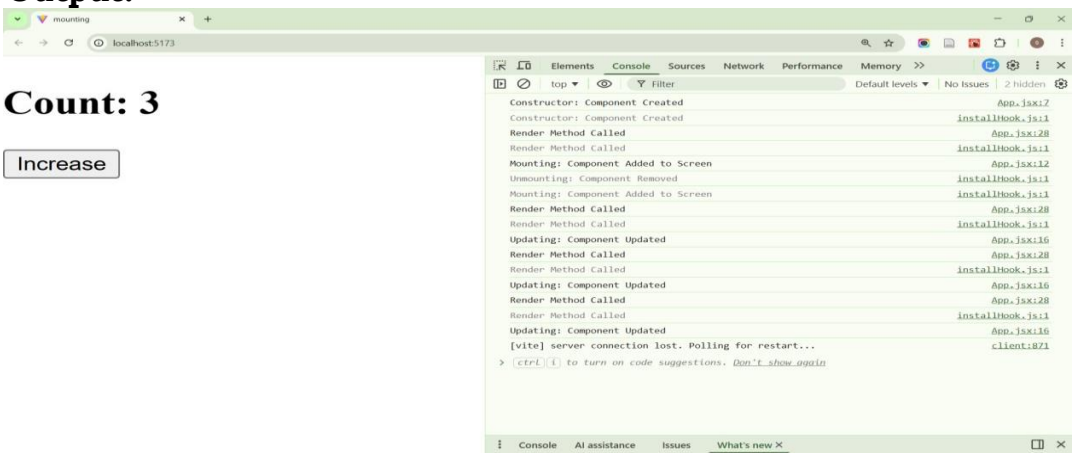
Example: simple example showing all three lifecycle phases in React.

App.jsx ● # index.css ×

App.jsx > ...

```
1  import React, { Component } from "react";
2
3  class Demo extends Component {
4
5    constructor(props){
6      super(props);
7      console.log("Constructor: Component Created");
8      this.state = { count: 0 };
9    }
10
11   componentDidMount(){
12     console.log("Mounting: Component Added to Screen");
13   }
14
15   componentDidUpdate(){
16     console.log("Updating: Component Updated");
17   }
18
19   componentWillUnmount(){
20     console.log("Unmounting: Component Removed");
21   }
22
23   increase = () => {
24     this.setState({count: this.state.count + 1});
25   }
26
27   render(){
28     console.log("Render Method Called");
29
30     return(
31       <div>
32         <h2>Count: {this.state.count}</h2>
33         <button onClick={this.increase}>Increase</button>
34       </div>
35     );
36   }
37 }
38
39 export default Demo;
40
```

Output:



5.What is JSX? Write a code snippet demonstrating how to embed JavaScript expressions and attributes within JSX?
Ans:

JSX (JavaScript XML) is a syntax extension for JavaScript recommended for use with React to describe what the user interface (UI) should look like. It allows developers to write familiar HTML-like markup directly within JavaScript code, which enhances code readability and simplifies UI development.

Core Concepts and Usage

- **HTML-like Syntax:** JSX looks very similar to HTML, using tags and attributes to define the UI structure within the same file as the rendering logic.
- **Transpilation:** Browsers cannot read JSX directly. During the build process, a transpiler like Babel converts the JSX code into standard JavaScript function calls, specifically React.createElement() calls, which the browser can understand.
- **React Elements:** JSX expressions are essentially syntactic sugar that evaluate to plain JavaScript objects called "React elements." These objects describe the structure of the UI on the virtual DOM, which React uses to efficiently update the real DOM.
- **Component Structure:** In React, components are functions or classes that return JSX, enabling a modular, component-based approach to building UIs.

Key Syntax Rules

- **Single Root Element:** A component must return a single top-level element. To group multiple elements, wrap them in a or a <>...</> (Fragment) to avoid adding unnecessary nodes to the DOM.

- **Close All Tags:** All tags, including typically self-closing HTML elements like <img> and <input>, must be explicitly closed with a trailing slash (e.g., <img src="..." />).

- **camelCase Attributes:** Many HTML attributes are written in camelCase in JSX to align with JavaScript property naming conventions.
    - 'class' becomes 'className'.
    - 'for' becomes 'htmlFor'.
    - 'onclick' becomes 'onClick'.
- **Embed JavaScript with Curley Braces:** You can embed any valid JavaScript **expression** (variables, function calls, arithmetic operations, etc.) within JSX by wrapping it in curly braces {}.
- *Example:* const name = 'Sara'; <h1>Hello, {name}</h1>.
- **Component Capitalization:** User-defined components must start with a capital letter (PascalCase) to distinguish them from built-in HTML tags (lowercase).

- *Example:* <MyComponent /> vs. .

Benefits

- **Readability:** Blending markup and logic makes the code more intuitive and easier to read and maintain.
- **Dynamic Content:** Seamlessly embedding JavaScript expressions allows for dynamic and interactive user interfaces.
- **Safety:** By default, React DOM escapes values embedded in JSX, helping prevent Cross-Site Scripting (XSS) attacks.
- **Tooling Support:** Modern code editors and build tools offer excellent support for JSX, including syntax highlighting, linting, and autocomplete, which enhances the developer experience.

Car.jsx	**App.js**

function Car() {
  const hp = 218 * 1.36;
  return (
  <>
  <h1>My car</h1>
  It has {hp} horsepower
//variable attached
 	
h1>Your car</h1>
 It has { 218 * 1.36;}
horsepower //Expression
attached
 	
  </>
  );
}
	import Car from './ Car;
	function App() {
	
	  return (
	
	  < Car />
	
	  );
	}
	
	export default App;

6.Describe the process of Props Communication between a parent and a child component with a code snippet?

Ans:

Props Communication

Props (short for properties) are used in React to pass data from one component to another. They allow components to communicate with each other and make components reusable.

What are Props?

Props are read-only inputs passed from a parent component to a child component.

They help in:

a. Passing **data**
b. Passing **functions**
c. Making **components reusable**
d. Maintaining **unidirectional data flow**

Key Points

e. Props are **immutable (cannot be changed)** by the child component.
f. Props are passed **from parent → child**.
g. Props help components become **dynamic and reusable**.

Props Communication Flow

React follows **Unidirectional Data Flow**.

Parent Component → Child Component (props)

Parent → Child (Direct Props): Most common method.

Child to Parent Communication:

In React, **data flow is normally one-way**: Parent → Child using **props**.
But sometimes a **child component needs to send data back to the parent** (for example: button click, form input, etc.). React handles this using a **callback function**. A callback function is a function passed as an argument to another function and executed later.

In React:

- Parent **creates a function**
- Parent passes the function to the child as props
- Child **calls the function**
- Parent **receives the data**

So, the parent function becomes a callback function.

App.js (Main Component)

```
Parent.jsx   Child.jsx   App.jsx
src > App.jsx > ...
1  import Parent from "./Parent";
2
3  function App(){
4    return <Parent/>;
5  }
6
7  export default App;
8
9
```

React needs a main/root component, which is usually App.jsx.
App.jsx is the root component in a React application. It acts as the main component where other components are rendered. The application starts from main.jsx, which renders App.jsx.

Parent.jsx

```
Parent.jsx   Child.jsx   App.jsx
src > Parent.jsx > ...
1  import Child from "./Child";
2
3  function Parent(){
4
5    function receive(msg){
6      alert(msg);
7    }
8
9    return (
10     <div>
11       <Child send={receive}/>
12     </div>
13   );
14 }
15
16 export default Parent;
17
18
```

Child.jsx

```
Parent.jsx   Child.jsx   App.jsx
src > Child.jsx > ...
1  function Child(props){
2
3    return (
4      <button onClick={()=>props.send("Hello Parent")}>
5        Send Message
6      </button>
7    );
8  }
9
10 export default Child;
11
12
```

Complete Flow

main.jsx

|

▼ **App.jsx**

|

▼

Parent.jsx

|

▼

Child.jsx

Output:

MODULE IV
Short Answers

1. How do you add state management capabilities to a functional React component?

Answer:

- In a functional component, we don't have a 'this' object or a 'setState' method like we do in classes.
- Instead, we "hook" into React's internal state engine using specific functions like

(1) Local State with useState:

- The useState hook is the bread and butter of React. It allows you to track strings, numbers, booleans, or objects that trigger a re-render when they change.

- **Syntax:** const [state, setState] = useState(initialValue);

(2) Complex State with useReducer:

- The useReducer hook in React is an alternative to useState that helps manage complex state logic.
- It is particularly useful when the state transitions are intricate, the next state depends on the previous one, or you need to manage multiple related state variables that change in concert

- **Syntax:** const [state, dispatch] = useReducer(reducer, initialState)

(3) Global State with useContext:

- The useContext hook in React is a powerful, built-in mechanism that allows functional components to subscribe to **contextual data** (a form of global state) and avoid **prop drilling**.
- It enables data to be passed through the component tree without manually passing props at every level.

- **Proceess of useContext:**
  const MyContext = createContext(); //create context
  <MyContext.Provider value={data}> //providing context
  const data = useContext(MyContext); //using the context

2. What constitutes a side effect, and how can you handle it in functional components in React?

Answer:

- A pure React component should be a function of its props and state: we give it inputs, and it returns JSX.
- A side effect is when that function reaches out to the world to change something else or fetch data.
- Side Effect is a task doesn't directly involve calculating the UI, it's likely a side effect. Common examples include:
  - **Data Fetching:** Making an API call to a server.
  - **Manual DOM Manipulation:** Changing the document title or adding a class to the body tag.
  - **Subscriptions:** Setting up a WebSocket connection or a timer (setTimeout/setInterval).
  - **Logging:** Sending analytics or console logs.
- In functional components we handle by use of the useEffect hook to manage these actions. It ensures that the side effect runs after the browser has finished painting the screen, so it doesn't block the user interface.

3. What is the role of keys in React's reconciliation (diffing) algorithm?

Answer:

React compares the old virtual DOM tree with the new one. When it hits a list, it iterates over both simultaneously.

- **Without Keys:** React matches items by **index** (order). If we insert an item at the top of a list of 1,000 items, React sees that "index 0" has changed, "index 1" has changed, and so on ,effectively re-rendering or updating the entire list because it thinks every item changed.

- **With Keys:** React matches items by **identity**. If we insert an item at the top, React sees that the existing keys still exist elsewhere and simply moves them in the DOM, only rendering the one new element.

- During the diffing process, keys allow React to perform a **linear O(n)** conversion instead of a more complex tree comparison. This keeps the UI snappy even with large datasets.

4. **Explain the performance and functional tradeoffs between controlled and uncontrolled inputs in React?**

Answer:

1. Controlled Components

In a controlled component, the form data is handled by a React component. The input value is driven by **state**, and any changes are handled by an **onChange** callback.

- **Source of Truth:** React State.
- **Flow:** User types → onChange fires → State updates → Component re-renders → Input displays new value.

Tradeoffs

- **Pros (Functional):** Instant validation, conditional disabling of buttons, and synchronization between multiple inputs (e.g., a "Confirm Password" field that checks the "Password" field in real-time).
- **Cons (Performance):** Every single keystroke triggers a full component re-render. In very large, complex forms with dozens of inputs, this can lead to "typing lag" if not optimized with memoization.

2. Uncontrolled Components:

Uncontrolled components act more like traditional HTML forms. The DOM handles the form data itself. You use a **ref** to pull the value from the field only when you actually need it (like when clicking "Submit").

- **Source of Truth:** The DOM.
- **Flow:** User types (DOM updates automatically) → User clicks Submit → You pull the value via ref.current.value.

Tradeoffs:

- **Pros (Performance):** Zero re-renders while the user is typing. This makes it significantly faster for massive forms or performance-constrained environments.
- **Cons (Functional):** It is harder to implement "live" features. You cannot easily validate a field as the user types or clear the input programmatically without manually touching the DOM via the ref.

5. **What are the cleanup steps for useEffect Hook to prevent memory leaks?**

Answer:

In React, the "cleanup" function is a mechanism to tidy up after a side effect once it's no longer needed. If you don't clean up, you risk **memory leaks**, where your application continues to perform tasks (like listening for events or running timers) for components that have already been destroyed.

The cleanup step is simply a **function returned** from your useEffect callback.

- React executes the cleanup function in two specific scenarios:
  Before the component unmounts (is removed from the DOM).
  Before running the effect again (if a dependency in the array has changed).

Long Answers

1. **Explain the useReducer hook and how it differs from useState for complex state logic?**
   Answer:
   useReducer hook is a powerful alternative to useState when you have complex state logic that involves multiple sub-values or when the next state depends on the previous one. It's particularly useful when the state logic involves more than just simple state updates. Handling complex state logic with useReducer in React involves defining a reducer function that manages state transitions based on dispatched actions.

   The useReducer hook is a React function that manages state using a **reducer function**. It is heavily inspired by Redux and follows a very specific pattern.

   Syntax:
   const [state, dispatch] = useReducer(reducer, initialState);

   where
   1. **State:** The current data.
   2. **Action:** An object that describes what happened (e.g., { type: 'increment' }).
   3. **Dispatch:** The function you call to send an action to the reducer.
   4. **Reducer:** A pure function that takes the current state and the action, then returns the **new** state.

```
import React, { useReducer } from 'react';

function counterReducer(state, action)
{
  switch (action.type)
{
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return { count: 0 };
    default:
      throw new Error('Unknown action');
  }
}
const Reducer = () => {
  const [state, dispatch] = useReducer(counterReducer, { count: 0 });
```

```
  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
};
export default Reducer;
```

output:

2. **Demonstrate how to create and implement a Custom Hook to reuse logic across components?**
   Answer:

   Creating and implementing a Custom Hook in React allows you to reuse stateful logic across different components.

   The following example demonstrates how to create a useCounter hook and use it in two separate components.

   Create a new file, for example, useCounter.js, to house the custom hook logic. This hook manages the counter state and related functions.

```
// useCounter.js
import { useState } from 'react';

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(c => c + 1);
  const decrement = () => setCount(c => c - 1);
  const reset = () => setCount(initialValue);
```

```
  // Return the state and functions you want to expose to components
  return {
    count,
    increment,
    decrement,
    reset,
  };
}

export default useCounter;
```

- **Now import and use the useCounter hook in any functional component.**

  Component 1: CounterComponentA.js

```
// CounterComponentA.js
import React from 'react';
import useCounter from './useCounter'; // Adjust the path as necessary

function CounterComponentA() {
  // Use the custom hook to get the count state and functions
  const { count, increment, decrement, reset } = useCounter(10); // Start at
10

  return (
    <div>
      <h2>Counter A: {count}</h2>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

export default CounterComponentA;
```

Component 2: CounterComponentB.js

```
// CounterComponentB.js
import React from 'react';
import useCounter from './useCounter'; // Adjust the path as necessary

function CounterComponentB() {
  // Use the custom hook again, it creates a separate instance of the logic
  const { count, increment } = useCounter(0); // Start at 0

  return (
    <div>
      <h2>Counter B: {count}</h2>
      <button onClick={increment}>Increment B</button>
    </div>
  );
}
```

```
export default CounterComponentB;
```

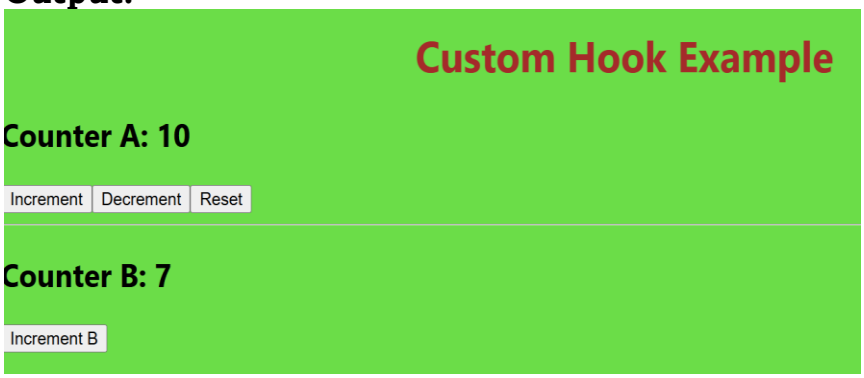
- **Use both components in your main application file.**

```
// App.js
import React from 'react';
import CounterComponentA from './CounterComponentA';
import CounterComponentB from './CounterComponentB';

function App() {
  return (
    <div>
      <h1>Custom Hook Example</h1>
      <CounterComponentA />
      <hr />
      <CounterComponentB />
    </div>
  );
}

export default App;
```

Output:



3. **Describe the useEffect Hook in detail, specifically how the dependency array controls the execution of side effects?**
   Answer:

   The useEffect Hook in React's documentation allows to perform side effects in functional components.
   It accepts two arguments: a function containing the side effect logic, and an optional dependency array that controls when that function runs.

   The primary purpose of useEffect is to handle operations that interact with the outside world or the browser APIs (e.g., data fetching, manual DOM manipulation, subscriptions, timers).
   syntax:

```
useEffect(() => {
  // Side effect logic here
  return () => {
    // Optional cleanup function here
  };
```

```
}, [dependency1, dependency2]);
```

There are Three "Modes" of the Dependency Array
How you define that second argument completely changes the behavior of your component.

1. No Dependency Array
If you omit the array entirely, the effect runs after **every single render**.

```
useEffect(() => {
  console.log("I run on every render!");
});
```

Usually used for global logging or synchronization that must be perfect, but it can cause massive performance issues if the effect is heavy.

2. The Empty Array [ ]
The effect runs once, immediately after the initial mount. It never runs again for the life of the component.

```
useEffect(() => {
  console.log("I only run once (on mount).");
}, [ ] ) ;
```

Fetching initial data from an API or starting a timer.

3. Array with Dependencies [data]
The effect runs on mount and then **only if the value of data has changed** since the last render.

```
useEffect(() => {
  console.log("I run when 'userId' changes.");
}, [userId]);
```

Refetching a user profile when the ID in the URL changes.

4. Discuss **Conditional Rendering** techniques in React (e.g., Ternary operators, Logical &&)?
   Answer:

   Conditional rendering in React allows you to selectively display UI elements based on specific conditions, such as user authentication, data loading states, or application permissions. Because React handles logic using standard JavaScript, we can use several native syntax options to manage these transitions.

   1. if/else Statements
   Standard JavaScript if/else statements can be used outside of the JSX return statement to decide which elements to render. This is ideal for handling complex conditions or logic that is too verbose for inline methods.

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h1>Welcome back!</h1>;
  } else {
    return <h1>Please sign up.</h1>;
  }
}
```

2.Ternary Operator (? :)

The conditional (ternary) operator is a concise way to handle if/else conditions inline within JSX.

```
import React from 'react';

function App() {
  const isLoggedIn = true;

  return (
    <div>
      {isLoggedIn ? (
        <h1>Welcome back!</h1> // Rendered if isLoggedIn is true
      ) : (
        <h1>Please log in.</h1> // Rendered if isLoggedIn is false
      )}
    </div>
  );
}

export default App;
```

3. Logical AND Operator (&&)

The logical AND operator (short-circuit evaluation) allows you to conditionally render an element or component when the condition is true, otherwise rendering nothing (effectively null or false).

```
import React from 'react';

function App() {
  const isMailboxEmpty = false;

  return (
    <div>
      <h1>Hello!</h1>
      {!isMailboxEmpty && (
        <p>You have 5 unread messages.</p> // Renders only if
isMailboxEmpty is false
```

```
  )}
    </div>
  );
}

export default App;
```

4. Switch Statements

n React, switch statements are commonly used for **conditional rendering** when you have multiple potential views or components to display based on a single state.

Because switch is a **statement** and not an expression, it cannot be used directly inside the return's JSX { } braces.

```
import React, { useState } from "react";

function App() {
  const [status, setStatus] = useState("loading");

  // Helper function to handle the switch logic
  const renderContent = () => {
    switch (status) {
      case "loading":
        return <p>Loading data...</p>;
      case "success":
        return <h1>Data Loaded!</h1>;
      case "error":
        return <p style={{ color: "red" }}>Error loading data.</p>;
      default:
        return null;
    }
  };

  return (
    <div>
      {renderContent()} {/* Call the function here */}
      <button onClick={() => setStatus("success")}>Show Success</button>
      <button onClick={() => setStatus("error")}>Show Error</button>
    </div>
  );
}

export default App;
```

5. **Discuss Event Handling Techniques in React. How do you pass arguments to an event handler without triggering it on render?**
Answer:

Event handling in React is similar to HTML but uses **camelCase** for event names (e.g., onClick instead of onclick) and passes a **function** reference rather than a string. React uses a **synthetic event system** to ensure consistent behavior across different browsers.

Event Handling Techniques in React

1.Passing a Function Reference: The most basic technique is to pass the function's name (reference) directly to the event handler prop. The function is only called when the event occurs.

```
function handleClick() {
  alert('Button clicked!');
}
<button onClick={handleClick}>Click Me</button>
```

2.Inline Functions (Arrow Functions): You can define an event handler inline using an arrow function. This is convenient for short functions or when you need to access component props/state within the handler.

```
function MyComponent({ value }) {
  return (
    <button onClick={() => alert(`Value is: ${value}`)}>
      Click Me
    </button>
  );
}
```

3.Class Component Method Binding: In class components, methods are not bound to the instance by default in JavaScript. To ensure this refers to the component instance inside the handler, you can bind it in the constructor or use class fields syntax (arrow functions as class properties).

```
class EventBind extends React.Component {
  constructor(props) {
    super(props);
    this.handleClick = this.handleClick.bind(this);
  }
  handleClick() {
    // 'this' is bound correctly
  }
```

```
  render() {
    return <button onClick={this.handleClick}>Click</button>;
  }
}
```

Passing Arguments to an Event Handler Without Triggering it on Render

To pass arguments without executing the function immediately during the render cycle, you must **pass a function that calls your handler**, rather than calling the handler directly.

- **Arrow Functions (Recommended):** Wrap the function call inside an anonymous arrow function. This creates a new function that only executes when the event is triggered.

  Example: <button onClick={() => handleClick(id)}>Click Me</button>

- **The .bind() Method:** Use the JavaScript bind method to create a new function with the arguments pre-applied. The first argument to bind sets the this context (usually null or this), and subsequent arguments are passed to the handler.

  Example: <button onClick={handleClick.bind(null, id)}>
  Click Me</button>.

- **Currying (Higher-Order Functions):** Define a function that returns another function. This allows you to "pre-load" the argument during render and return the actual handler to React.

  Example: const handleClick = (id) => (event) => { console.log(id); };

6. What are the best practices for handling form submission and input data in React Explain with Suitable Example?
Answer:

In React, handling form data effectively centers n predictability and validation. The industry standard is moving toward Uncontrolled Components with libraries like React Hook Form for performance, though Controlled Components remain the fundamental pattern for simple forms.

1. Controlled Components (The Standard Approach)

In this pattern, React state is the "single source of truth." Every keystroke updates the state, and the input value is driven by that state.

```
import React, { useState } from 'react';

const SimpleForm = () => {
  const [formData, setFormData] = useState({ username: '', email: '' });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevents page reload
    console.log("Submitted Data:", formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="username" value={formData.username}
onChange={handleChange} />
      <input name="email" value={formData.email}
onChange={handleChange} />
      <button type="submit">Submit</button>
    </form>
  );
};
```

2.Uncontrolled Components (Ref-Driven)

In this pattern, the DOM handles the form data internally. React "pulls" the values only when needed (usually on submission) using refs

```
import React, { useRef } from 'react';

// Uncontrolled Component Example
function UncontrolledForm() {
  const inputRef = useRef();

  return (
    <form onSubmit={ (e) => { e.preventDefault();
console.log(inputRef.current.value); } }>
      <input type="text" ref={ inputRef } />
      <button type="submit">Submit</button>
    </form>
  );
}
```

3.Example with Validation (React Hook Form)
This approach is more performant because it doesn't trigger a re-render on every keystroke.

```
import { useForm } from "react-hook-form";

export function ModernForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (data) => console.log(data);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <label>Email</label>
      <input {...register("email", { required: "Email is required" })} />
      {errors.email && <span>{errors.email.message}</span>}

      <button type="submit">Log In</button>
    </form>
  );
}
```

MODULE-V
SHORT ANSWERS

1. **What is the role of BrowserRouter and Route in React Router?**
Answer:
BrowserRouter: It is a parent component that uses the HTML5 History API to keep your UI in sync with the URL. It wraps the entire application to enable routing.

Syntax:

```
import { BrowserRouter } from "react-router-dom";

    <BrowserRouter>

   {/* Components or Routes go here */}

</BrowserRouter>
```

- **Route:** It is a component used to define a mapping between a specific URL path and a React component. When the path matches the current URL, the specified component is rendered.

Syntax:

```
<Route path="URL_path" element={<ComponentName />} />
```

2. **How do you extract values using the useParams() hook?**

Answer:

The useParams() hook is used in functional components to access dynamic parameters from the current URL (defined as :paramName in the Route).

Implementation: You call the hook inside the component, which returns an object of key/value pairs.

Syntax:

```
import { useParams } from "react-router-dom";

const params = useParams();
```

Example: const { id } = useParams(); where id is the dynamic segment of the path.

Example:

```
import { useParams } from "react-router-dom";

function User() {

  const { id } = useParams();

  return <h2>User ID: {id}</h2>;

}
```

3. **Define the Context API and its role in global state management?**

Answer:

The **Context API** is a built-in React feature used to share data (state) across the entire component tree without having to pass props down manually through every level.

Role in Global State Management:

- It simplifies **Global State Management** by providing a central "store" (Provider) that any nested component (Consumer/useContext) can

access directly, effectively eliminating the need for complex state lifting.

- It helps to **avoid prop drilling** by making data available to any component in the application.

4. What is Client-Server Communication in the context of a React app?

Answer: It refers to the process where a React application (the **Client**) sends requests to a backend API or web server (the **Server**) to fetch, create, update, or delete data.

Mechanism: This is typically handled using the fetch() API or libraries like Axios, usually triggered within a useEffect hook to ensure data is synchronized with the UI.

5. What are Nested Routes, and when should they be used ?

Answer:

Nested Routes are routes defined inside another route, allowing a portion of the UI to change while the parent layout remains constant.

Syntax:

```
<Route path="/dashboard" element={<Dashboard />}>

  <Route path="profile" element={<Profile />} />

</Route>
```

Here:

- /dashboard → Displays **Dashboard component**
- /dashboard/profile → Displays **Profile component inside Dashboard**

Usage: They should be used when building complex layouts where a sub-view depends on a parent view (e.g., a "Dashboard" page with internal tabs like "Profile" and "Settings"). They use the <Outlet /> component to render the child content.

LONG ANSWERS

1. **Compare the Context API with the Reducer Pattern for managing advanced application state?**
   Answer:

Feature	Context API	Reducer Pattern
Definition	Context API is a React feature used to share data globally across components without passing props manually through every level.	Reducer Pattern is a state management technique that updates state using a reducer function based on dispatched actions.
Purpose	Mainly used to avoid **prop drilling** and provide global data such as theme, authentication, or user information.	Used to manage **complex state logic** in a predictable and organized way.
Working Principle .	Data is provided through a **Provider component** and accessed in child components using **useContext()**.	State changes are handled by a **reducer function** that receives the current state and an action, then returns a new state
State Management	Suitable for **simple or moderate global state management**.	Suitable for **complex state transitions and multiple related state updates**.
Data Flow	Data flows directly from the **Context Provider to consuming components**.	Data flows through **actions → reducer → updated state**
Code Complexity	Easy to implement with less code for small applications	Requires defining **actions, reducer functions, and dispatch**, making it slightly more complex.
Performance	When context value changes, all consuming components may re-render.	Reducer helps structure updates efficiently and can improve maintainability in large applications.
Usage Example	Theme management, authentication status, language settings.	Counters, form state management, complex UI state changes

Combination Usage	Often used together with the Reducer Pattern to create a powerful global state management system.	Frequently combined with Context API to manage and distribute complex application state.
Conclusion	Context API simplifies global data sharing across components.	Reducer Pattern provides structured and predictable state updates for complex applications.

2. Explain the implementation of Application Routing, including navigation components like Link and NavLink?

Answer:

Implementation of Application Routing in React

Application routing allows a React application to navigate between different pages/components without refreshing the browser. This is implemented using the library React Router.

Application routing in React is the process of mapping specific URL paths to React components, allowing users to navigate between different views in a Single Page Application (SPA) without reloading the page. It is primarily implemented using the **React Router** library (react-router-dom), which provides a declarative way to structure navigation.

1. Implementation Steps (React Router v6+)

To implement routing, you must install the library (npm install react-router-dom) and wrap your app in a router provider.

Step 1: Set up the Browser Router Wrap the root component with BrowserRouter to enable the HTML5 history API.

Step 2: Define Routes Use <Routes> as a container, and define individual paths using <Route>.

Step 3: Map Paths to Components Use the element prop to define which component should render at a given path

Example:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from './Home';
import About from './About';
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Working Flow

1. The application loads inside **BrowserRouter**.
2. **Routes** checks the current URL.
3. If the URL is /, the **Home component** is rendered.
4. If the URL is /about, the **About component** is rendered.

2. Navigation using Link

The **Link** component is used to navigate between routes without reloading the page.

Example:

```
import { Link } from "react-router-dom";

function Navbar() {

  return (

    <div>

      <Link to="/">Home</Link>

      <Link to="/about">About</Link>

    </div>

  );

}
```

Output:

Clicking **Home** opens the Home component.

3. Navigation using NavLink

The **NavLink** component is similar to Link but provides **active styling for the current route**.

Example:

```
import { NavLink } from "react-router-dom";

function Navbar() {

  return (

    <div>
```

```
      <NavLink to="/" className={({isActive}) => isActive ? "active" : ""}>
        Home
      </NavLink>
      <NavLink to="/about" className={({isActive}) => isActive ? "active" : ""}>
        About
      </NavLink>
    </div>
  );
}
```

Output:

When the user visits /about, the **About link becomes highlighted**.

3. Describe the general workflow for implementing user authentication in react application?
Answer:

The general workflow for implementing user authentication in a React application involves setting up a **backend service**, managing the **authentication state** using the React Context API or a library like the Auth0 React SDK, and controlling access to routes using React Router.
User authentication ensures that **only authorized users can access certain parts of an application**. In React applications, authentication is often implemented using **tokens such as JWT** with the help of routing libraries like React Router.
Implementing user authentication in a React application typically follows a workflow that coordinates the frontend (React), a backend (Node/Express or an external service), and a storage mechanism for user sessions.

1. **User Input and Credentials Submission**

   Create Login/Signup Forms: Use React components to capture user credentials (e.g., email and password).

   Handle Form Submission: Use asynchronous functions to send a `POST` request with these credentials to the backend API (e.g., using `fetch` or `Axios`)

2. **Backend Verification and Token Issuance**

   Validate Credentials: The backend verifies the credentials against the database.

Generate Tokens: If valid, the server creates a **JSON Web Token (JWT)** or a session cookie.

Respond to Client: The backend sends the token or sets a cookie in the response to the React application

3. **Client-Side Token Storage**

   Choose Storage Method:

   a. **localStorage**: Persists across tabs and browser restarts.

   b. **sessionStorage**: Clears when the browser tab is closed.

   c. **HTTP-Only Cookies**: More secure against Cross-Site Scripting (XSS) as they are inaccessible to JavaScript.

- **Save the Token**: Store the received JWT or session identifier to maintain the user's logged-in state

4. **Global State Management**

   Create Auth Context: Use the React Context API to manage a global `isAuthenticated` state and user profile data.

   Custom Hooks: Implement a `useAuth` hook to easily access authentication functions (login, logout) and the current user's status throughout the app.

5. **Routing and UI Protection**

   Protected Routes: Use React Router to create a wrapper component (e.g., ProtectedRoute) that checks if a user is authenticated before rendering a page.

   Redirects: Automatically redirect unauthenticated users to the login page when they attempt to access private routes.

   Conditional Rendering: Dynamically show or hide UI elements (like "Profile" links or "Logout" buttons) based on the authentication status.

6. **Authenticated API Requests**

   Authorization Header: For subsequent requests to protected API endpoints, attach the stored token in the `Authorization` header as a `Bearer <token>`.

   Intercept Responses: Use Axios Interceptors to handle common authentication errors, such as `401 Unauthorized`.

7. **Logout Flow**

   Clear Session: Remove the token from local/session storage and reset the global auth state.

   Redirect: Send the user back to the login or home page after the session is cleared.

4. Discuss Advanced state management using the context API to solve the prop drilling problem?

Answer:

The React **Context API** solves the "prop drilling" problem by providing a way to share data globally across the component tree without manually passing props through every intermediate component. This makes state management in deeply nested applications cleaner and more maintainable.

Prop Drilling Problem

Prop drilling occurs when data is passed from a parent component to deeply nested child components through multiple layers.

Example Problem Structure:

```
App
└── Parent
    └── Child
        └── GrandChild
```

If **GrandChild** needs data from **App**, the data must pass through **Parent** and **Child**, even though they don't use it.

Problems:

- Code becomes complex.
- Components become tightly coupled.
- Maintenance becomes difficult.

Understanding Prop Drilling

Prop drilling occurs when data is passed through multiple layers of components, even to components that don't need the data themselves, solely to reach a deeply nested component.

- **Code Clutter:** Intermediate components have unnecessary props, making the code harder to read and understand.
- **Maintenance Issues:** Changing the required props in a deeply nested component means updating every intermediate component in the chain, increasing maintenance effort.
- **Reduced Reusability:** Components become tightly coupled to the specific props they pass down, limiting their reusability in different parts of the application.

Advanced State Management with Context API

The Context API provides a direct channel to share values like user authentication status, themes, or application settings.

Core Mechanism

The Context API works through three main steps:

1. **Create a Context:** Use React.createContext() to create a context object, which comes with a Provider and a Consumer component.

```
import React, { createContext } from 'react';
const UserContext = createContext(defaultValue);
```

2. **Provide the Context:** Wrap the components that need access to the data with the Provider component and pass the state as a value prop. All components within the provider's scope can access this value.

```
function App() {
  const [user, setUser] = useState({ name: 'John Doe' });
  return (
    <UserContext.Provider value={{ user, setUser }}>
      <MainContent />
    </UserContext.Provider>
  );
}
```

3. **Consume the Context:** In any descendant component (no matter the nesting depth), use the useContext hook to access the provided value directly.

```
import { useContext } from 'react';

function Profile() {
  const { user } = useContext(UserContext);
  return <p>Welcome, {user.name}</p>;
}
```

Benefits over Prop Drilling

- **Direct Data Access:** Components can access the global state directly, eliminating the need for intermediate components to handle props they don't use.
- **Cleaner Codebase:** Reduces the number of unnecessary props passed through the component tree, leading to cleaner and more maintainable code.
- **Built-in Solution:** The Context API is a built-in React feature, so no external libraries are required for basic global state management.
- **Improved Scalability:** It helps manage application-wide data in a centralized manner, making the application more scalable as it grows.

5. Explain the Route Setup and Implementation process in React Router, including browser Router, Routes and Route?
Answer:

Routing in a React application allows navigation between different components without reloading the page. This is implemented using the library React Router. To define and implement routing within your React project, you utilize the <Route>

component provided by React Router. Each route specifies a particular URL path and the corresponding React component that should be rendered when the user navigates to that
path. The collection of routes is typically wrapped within a <Routes> component, which acts as a container for all individual route definitions. This setup allows for clear, declarative routing logic, making it easier to manage and scale your application's navigation structure.
Route Setup and Implementation
Steps:

1. Install React Router
2. Configure routes
3. Render components based on URL

BrowserRouter
BrowserRouter is the main routing component that wraps the entire application.
It enables routing by using the **browser's history API** and keeps the UI synchronized with the URL.

Syntax:
```
import { BrowserRouter } from "react-router-dom";
function App() {
  return (
    <BrowserRouter>
      {/* Routing components go here */}
    </BrowserRouter>
  );
```

```
}
```

Routes

Routes is a container component used to group multiple Route components. It checks all defined routes and renders the component whose path matches the current URL.

Syntax:

```
import { Routes } from "react-router-dom";
<Routes>
{/*Routecomponents*/}
</Routes>
```

Route

Route defines the mapping between a **URL path** and a **React component**. When the URL matches the specified path, the corresponding component is displayed.

Syntax:

```
import { Route } from "react-router-dom";
<Route path="/home" element={<Home />} />
<Route path="/about" element={<About />} />
<Routes>
  {/* Route components */}
</Routes>
```

Syntax:

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/about" element={<AboutPage />} />

  <Route path="/profile" element={<UserProfile />} />

</Routes>
```

Complete Example:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```